reverse the string without using any shortcut & in built function

Q -> remove duplicates from list list1 = [1,2,3,5,4,3,2,,5,2]

```
list1=[1,2,3,5,4,3,2,1,5,2]
```

reverse 3 digit number without using loops | no = 123

```
class Student :
  def show (self):
    print("I am in student class")

obj = Student()
obj.show()
Student().show()
```

```
I am in student class
I am in student class
```

```
class Student:
  def _init_(self):
    print("I am in default constructor")

  def show(self):
    print("I am in Student class")

obj=Student()
obj.show()
```

```
I am in Student class
```

```
class Student:
  def __init__(self):
    self.name="Insiya"
    self.age=18
    self.percentage=91
  def showInfo(self):
    print("Name:",self.name)
    print("Age :",self.age)
    print("Percentage :",self.percentage)
```

```
s=Student()
s.showInfo()
```

```
Name: Insiya
Age : 18
Percentage : 91
```

```
class Student:
  def __init__(self,name,age,percentage):
    self.name=name
    self.age=age
    self.percentage=percentage

  def showInfo(self):
      print("Name:",self.name)
      print("Age:",self.age)
      print("Percentage:",self.percentage)

s=Student("Joy",22,55)
s1=Student("Aditya",22,55)
s.showInfo()
s1.showInfo()
print("Name =",s.name)
```

```
Name: Joy
Age: 22
Percentage: 55
Name: Aditya
Age: 22
Percentage: 55
Name = Joy
```

```
class Student:
  def __init__(self,name,age,percentage):
    self.name=name
    self.age=age
    self.percentage=percentage

  def showInfo(self):
      print("Name:",self.name)
      print("Age:",self.age)
      print("Percentage:",self.percentage)

name=input("Enter Name\t:")
age=int(input("Enter Age\t:"))
percentage=float(input("Enter Percentage:"))
s=Student(name,age,percentage)
s1=Student(name,age,percentage)
s.showInfo()
s1.showInfo()
```

```
Enter Name      :Insiya
Enter Age       :18
Enter Percentage:91
Name: Insiya
Age: 18
Percentage: 91.0
Name: Insiya
Age: 18
Percentage: 91.0
```

```
class MyPython:
  def __init__(self):
    print("First Constructor")
  def __init__(self):
    print("Second Constructor")

m=MyPython()
#constructor overloading is not allowed in python, if u try to overload the cor
```

```
Second Constructor
```

delete instance variable

```
class MyPython:
  def __init__(self):
    self.a = 10
    self.b = 20
    self.c = 30
    self.d = 40

  def func(self):
    del self.d

m=MyPython()
print(m.__dict__)
m.func()
print(m.__dict__)
del m.c
print(m.__dict__)
```

```
{'a': 10, 'b': 20, 'c': 30, 'd': 40}
{'a': 10, 'b': 20, 'c': 30}
{'a': 10, 'b': 20}
```

```
class MyPython:
    def __init__(self):
        print("Constructor is called")
```

```
    def __del__(self):
        print("Destructor is called")

    def func(self):
        print("This is a function")

obj = MyPython()
obj.func()
del obj
```

```
Constructor is called
This is a function
Destructor is called
```

Static Variable

```
class MyPython:
  x=20
  def __init__(self):
    self.y=10

m1=MyPython()
m2=MyPython()
print("m1 -->",m1.x,m1.y)
print("m2 -->",m2.x,m2.y)
MyPython.x=111
m1.y=222
print("m1 -->",m1.x,m1.y)
print("m2 -->",m2.x,m2.y)
```

```
m1 --> 20 10
m2 --> 20 10
m1 --> 111 222
m2 --> 111 10
```

local variables

```
class Test:
    def func1(self):
        a = 100
        print(a)

    def func2(self):
        b = 222
        print(b)
```

```
t = Test()
t.func1()
t.func2()
```

```
100
222
```

Accept 5 paper marks, rollno, name;

and calculate total & percentage if percentage >=85, student can take admission else cant take asmission

using class & object & constructor

```
class A:
  def showA(self):
    print("showA function")

class B(A):
  def showB(self):
    print("showB function")

obj=B()
obj.showA()
obj.showB()
```

```
showA function
showB function
```

```
class A:
    def showA(self):
        print("showA function")

class B(A):
    def showB(self):
        print("showB function")

class C(B):
    def showC(self):
        print("showC function")

obj = C()
obj.showA()
obj.showB()
obj.showC()
```

```
showA function
showB function
```

```
showC function
```

```
class Parent :
  def bike(self):
      print("splender +")

class Child(Parent):
  def bike(self):
      print("HB")

obj=Child()
obj.bike()
```

```
HB
```